

REPORT

Master Embedded Systems

Distributed Embedded Systems

Round-Trip Measurement of Communication Duration

By: Jefferson Janisch, Bsc
Student Number: 2310297014

By: Alex Patai, Bsc
Student Number: 2310297015

Supervisor: Dr. Dipl.-Ing. Dietmar Millinger

Vienna, 04.05.2025

Table of Contents

1	Introduction	2
1.1	Code Snippets of the Master.....	4
1.2	Code Snippet of the Client	6
2	50 000 Measurements	6
2.1	Normal operation with Nice Level not Set.....	7
2.2	Normal operation with Nice Level Set to the Maximum	7
2.3	Measurements with High CPU Load	8
3	500 000 Measurements	10
3.1	250 000 Measurements without Nice Level.....	10
3.2	250 000 Measurements with Nice Level.....	11
	List of Figures.....	13
	List of Tables.....	14

1 Introduction

Team 3, with members Jefferson Janisch and Alex Patai, has chosen **Rust** as the programming language for the implementation of the round-trip measurement task. The setup, as the exercise description specified, consisted of two Raspberry Pi 4B nodes and a general-purpose laptop, all placed on the same local network connected by a Netgear GS308 switch. The following figure depicts the used network.

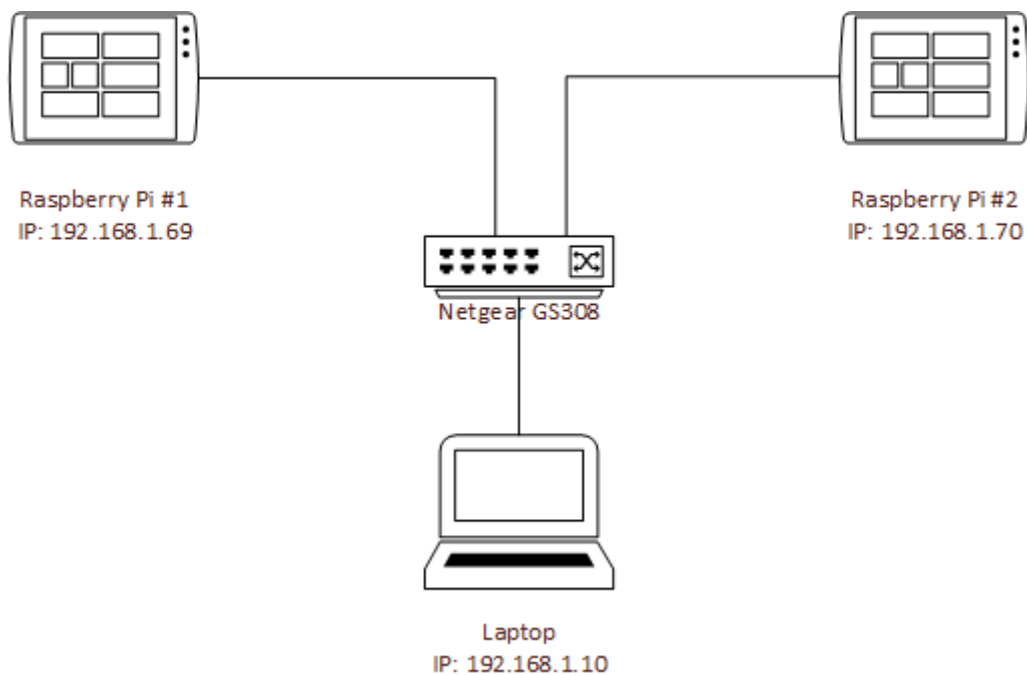


Figure 1: Network setup

The two Raspberry Pi modules were assigned the following IP addresses:

Table 1: IP addresses

Raspberry Pi ID	Team Member	IP Address
#1	Jefferson Janisch	192.168.1.69
#2	Alex Patai	192.168.1.70

The subnet mask 255.255.255.0 has been used consistently throughout the implementation. The final setup is shown in Figure 2.

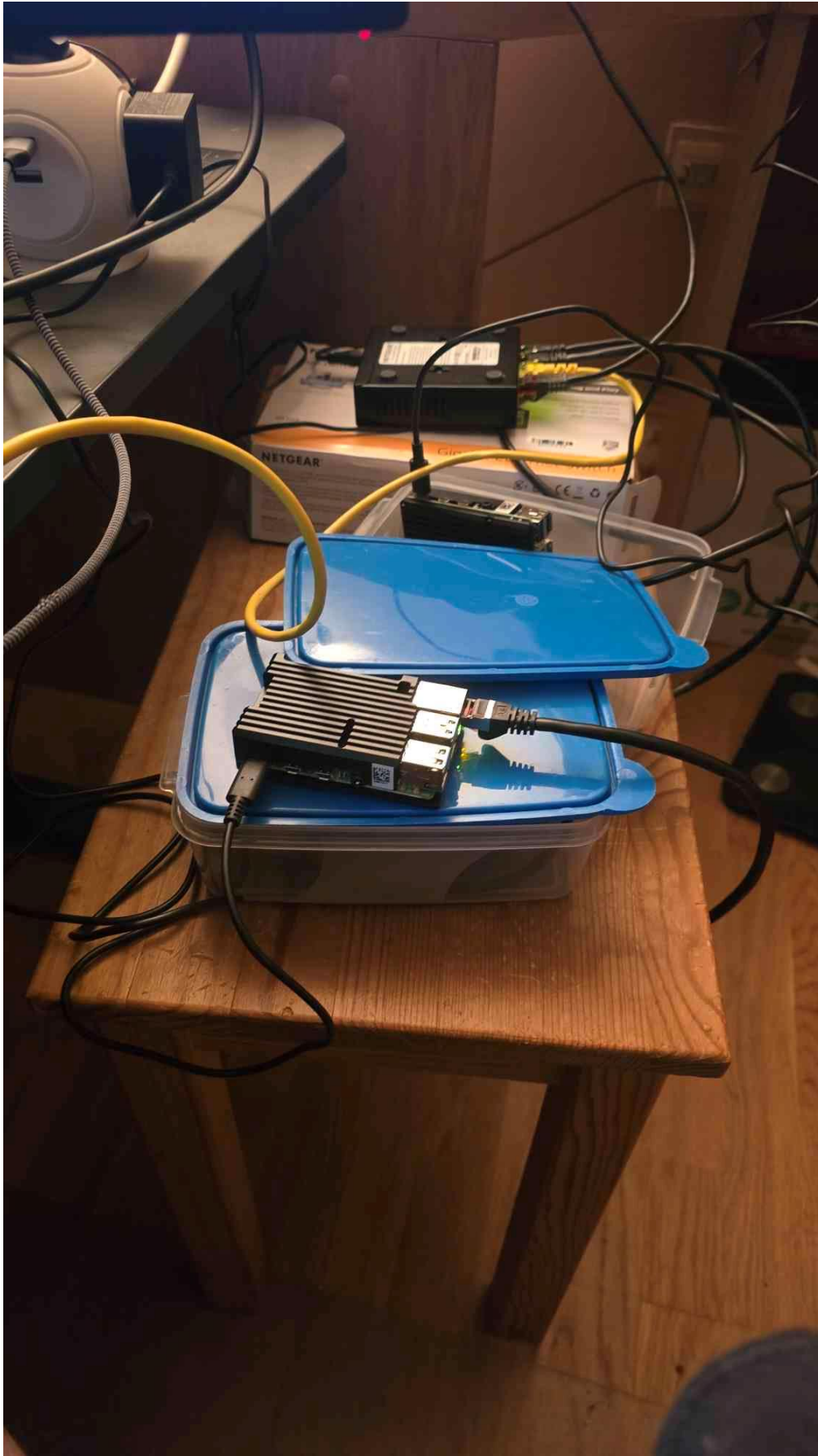


Figure 2: Assembled setup

1.1 Code Snippets of the Master

In our software, since it is written in Rust, the Rust conventions have been used to handle UDP connections.

The client and the master, both use the same function to handle their UDP connections, but they are compiled conditionally either as the „Side::Master“ or the „Side::Client“. If the master is compiled the following code is part of the **handle_connections()** function:

```

let socket = UdpSocket::bind(IP_MASTER)?;
    println!("Master bound to {:?}", IP_MASTER);

    let mut buffer = [0u8; BUFF_SIZE];

    let mut measurements: [(DateTime<Local>, DateTime<Local>);
ROUNDTRIP_COUNT] = [(Default::default(), Default::default());
ROUNDTRIP_COUNT]; // = //[(TimeDelta::default(), TimeDelta::default());
ROUNDTRIP_COUNT];

    for i in 0..ROUNDTRIP_COUNT {

        let start = chrono::Local::now();

        socket.send_to(format!("{i}").as_bytes(), IP_ECHO)?;

        socket.recv_from(&mut buffer)?;

        let end = chrono::Local::now();

        measurements[i] = (start, end);

        // println!("Received response: {}",
String::from_utf8_lossy(&buffer[..number_of_bytes]));

    }

    let mut measurement_file =
File::create_new("50_000_high_CPU_load.csv").unwrap();

    writeln!(measurement_file, "{},{},{", "Start - HH:MM:SS:NANOSECONDS",
"End - HH:MM:SS:NANOSECONDS", "Difference in NANOSECONDS").unwrap();

    measurements.iter().for_each(|(start, end)|{
        writeln!(measurement_file, "{},{},{", start.format("%T:%f"),
end.format("%T:%f"), (*end-*start).num_nanoseconds().unwrap()).unwrap();
    });

    return Ok(());

```

This function opens the UDP socket and once the socket is bound (connection is established) it creates the buffer which holds the round-trip measurements. The measurements are stored as tuples of two `DateTime<Local>` objects, which store the local time to a nanosecond granularity. Afterwards, the round-trip measurements are executed, by calling `chrono::Local::now()` before transmitting a message and after receiving one. Once the for-loop terminates the measurements and their differences are written to a newly created file, in the snippet provided, to „50_000_high_CPU_load.csv“ in a comma-separated-value format.

1.2 Code Snippet of the Client

As mentioned before, the client also utilizes the `handle_connection()` function, but conditionally compiled as the client side („Side::Client“). The `handle_connection()` function of the client looks as follows:

```
let socket = UdpSocket::bind(IP_ECHO)?;
    println!("Echo bound to {}", socket.local_addr()?);

    println!("Waiting for Master to send first message ...");

    let mut buffer = [0u8; BUFF_SIZE];

    loop {
        // Receive data from the client
        let (number_of_bytes, src_addr) = socket.recv_from(&mut buffer)?;
        // println!("Received '{}' from {}",
String::from_utf8_lossy(&buffer[..number_of_bytes]), src_addr);

        // Optionally send a response back
        socket.send_to(b"pong", src_addr)?;
    }
```

The client once connected to the master, waits for bytes, sent by the master and once it receives them, sends the response „pong“ back. The client remains in an infinite loop until the master break the connection, in which case `socket.recv_from()` function will terminate the loop with an exception.

2 50 000 Measurements

It proven impossible to generate network load, so our solution has taken a different approach. Instead of the increased network load, we have taken into account the Linux scheduling when

the measurement process is created. To influence this, the Linux **Nice Level** has been set in specific cases, providing a higher priority to the process when it comes to scheduling.

2.1 Normal operation with Nice Level not Set

Our measurements for the normal operation of the round trip measurement software yielded the minimum, maximum and mean values, shown in the following table:

Table 2: Minimum, maximum and mean values of 50.000 measurements for normal operation without Nice Level priority

Normal operation without Nice Level	Min	131 μ s
	Max	1213 μ s
	Mean	151.62 μ s

Figure 3 shows the distribution of the acquired measurements, with over 12000 messages returning in about 142 milliseconds and cumulatively about 20000 messages doing so under 150ms.

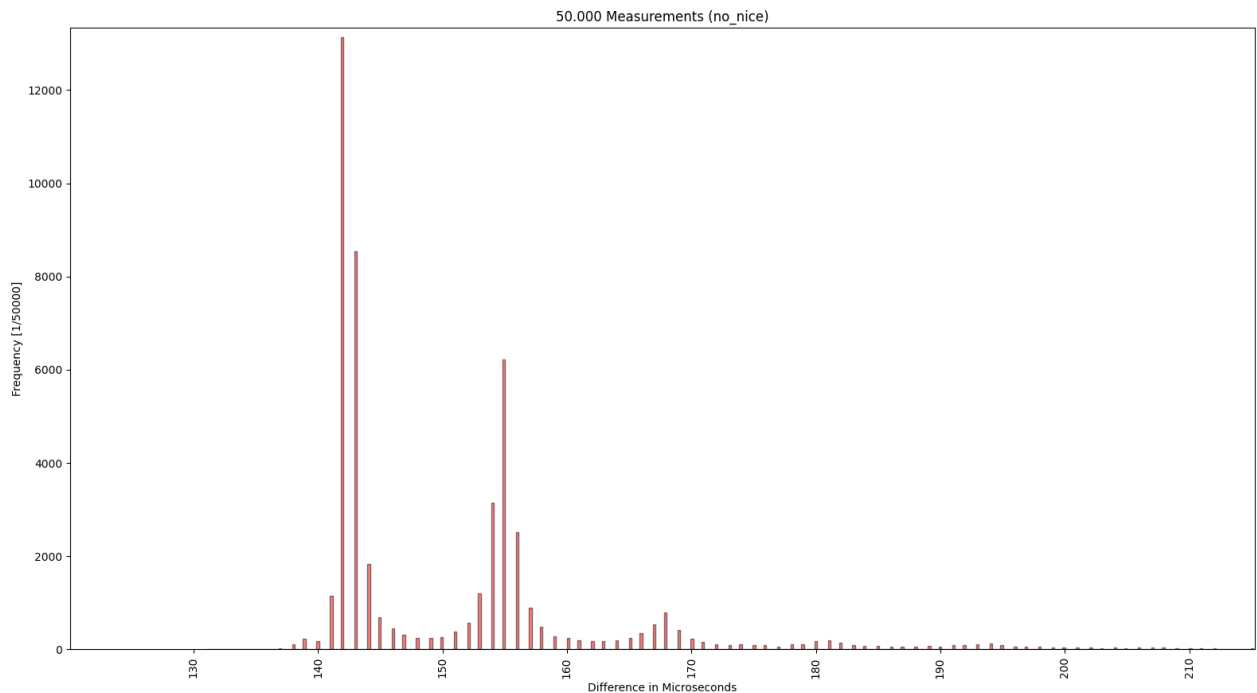


Figure 3: 50.000 measurements without Nice Level priority

2.2 Normal operation with Nice Level Set to the Maximum

The measurements for the normal operation with the Nice Level set to the maximum was executed with the expectation that the Linux scheduler has a major impact on the timeliness

of the communication. The acquired minimum, maximum and mean values are shown in Table 3.

Table 3: Minimum, maximum and mean values of 50.000 measurements for normal operation with Nice Level priority

Normal operation with Nice Level	Min	137 μ s
	Max	1009 μ s
	Mean	149.32 μ s

Although the minimum value proves to be higher than that of the case without the Nice Level set, the maximum value is more than 200 milliseconds lower and the mean is round-trip is also by 2 milliseconds shorter.

The histogram, showing the distribution of the measurements is shown in Figure 4.

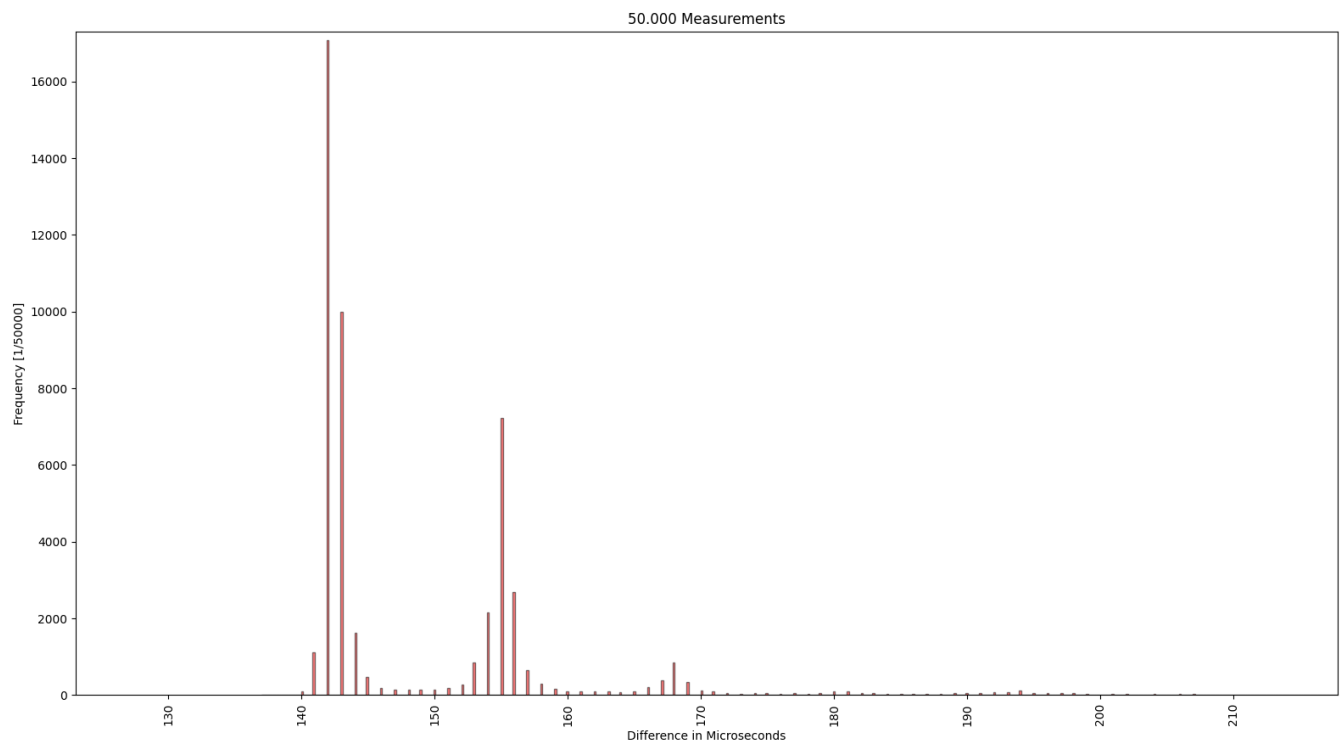


Figure 4: 50.000 measurements with Nice Level priority

2.3 Measurements with High CPU Load

Measurements with high CPU load show a great difference to the measurements under normal conditions.

To achieve a high CPU load, the terminal command “yes &> /dev/null &”, which occupies the CPU by writing “yes” to “/dev/null” continuously. The ampersand character ensures that the

task is sent to the background. The above-mentioned command has been repeatedly executed 10 times, which caused a significant and observable degradation of the system's processing speed.

Table 4: Minimum, maximum and mean values of 50.000 measurements for normal operation with high CPU load

High CPU load	Min	127 μ s
	Max	21854 μ s
	Mean	2136.36 μ s

Table 4 shows, that the minimum round-trip time was not impacted by the high CPU load, but the maximum round-trip amounted to 21854 microseconds, and the mean round-trip across 50.000 measurements has increased by about 15x.

The distribution of the round-trip times for this case is shown in Figure 5.

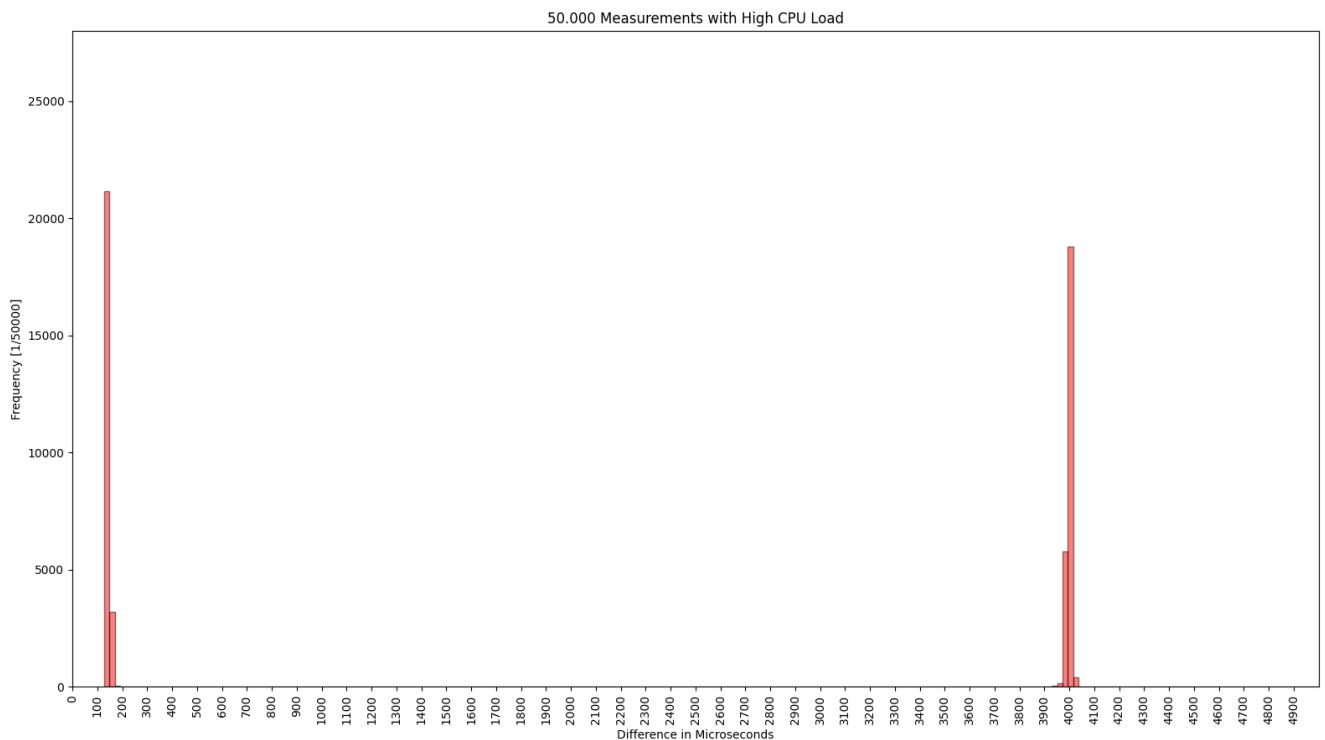


Figure 5: 50.000 measurements with high CPU load

The distribution of the round-trip times show a strong separation between quite low (120-180 μ s) and quite high (~4ms) measurements. The explanation for this is the CPU being the bottleneck, does not get to process every message in time.

3 500 000 Measurements

Writing our software in Rust, it has proven impossible to create buffers as large as 500.000 elements. Unfortunately, we had to re-define the task to only gather 250.000 measurements, and similarly to the previous measurements, we do these for both the Nice Level optimized processes and for those without.

3.1 250 000 Measurements without Nice Level

Our measurements for the 250.000 round trip measurement without Nice Level optimization yielded the minimum, maximum and mean values, shown in the following table:

Table 5: Minimum, maximum and mean values of 250.000 measurements for normal operation without Nice Level priority

250.000 measurements without Nice Level	Min	116 μ s
	Max	610 μ s
	Mean	146.62 μ s

The resulting measurements show significantly lower values than the measurements, where only 50.000 data points were gathered.

The distribution of the data points is shown in Figure 6.

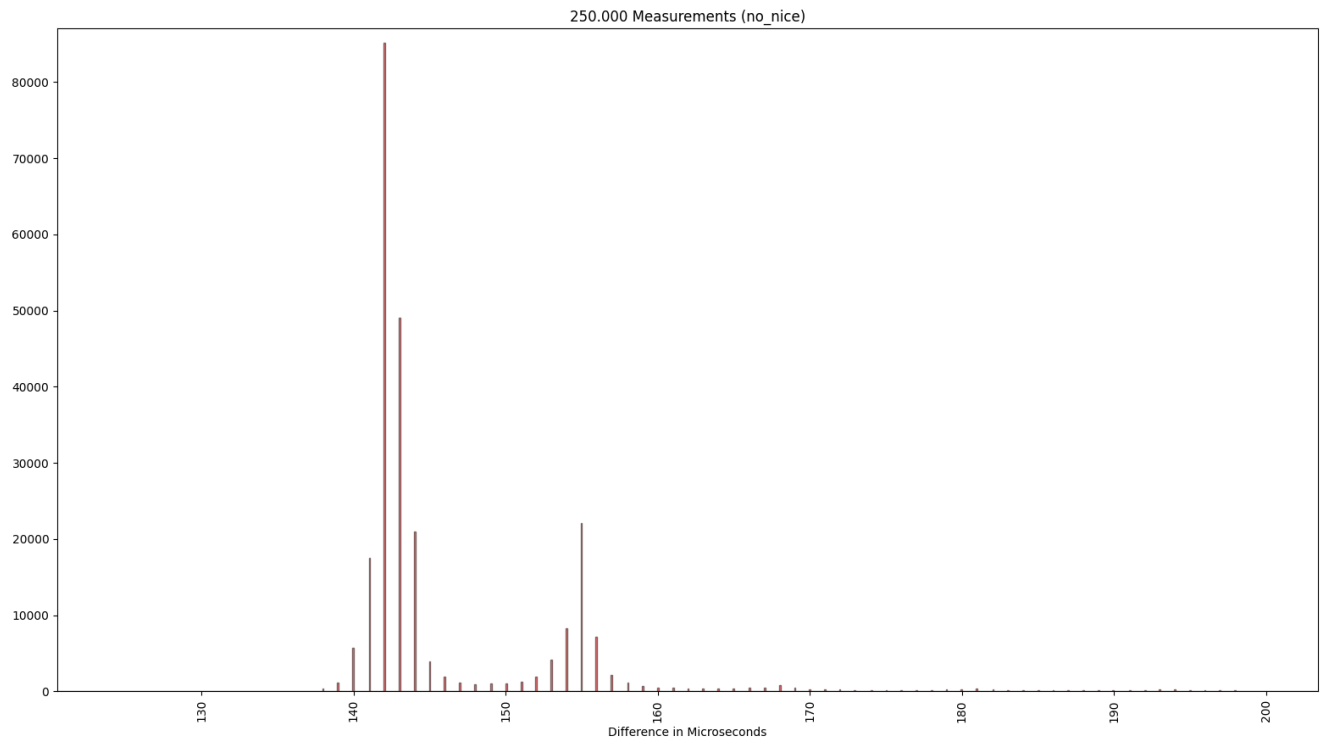


Figure 6: 250.000 measurements without Nice Level priority

The distribution of the measurements shows no significant outliers, with the most round-trip being completed between 140 and 150 μs .

3.2 250 000 Measurements with Nice Level

The measurements for the normal operation with the Nice Level set to the maximum was executed with the expectation of seeing improvement in the round-trip times compared to the normally executed measurements once again. The acquired minimum, maximum and mean values are shown in Table 6.

Table 6: Minimum, maximum and mean values of 250.000 measurements for normal operation without Nice Level priority

250.000 measurements with Nice Level	Min	100 μs
	Max	3348 μs
	Mean	145.79 μs

Table X shows, that although the minimum round-trip time has been decreased, the maximum is increased to about 5x. The justification for this could be one or more background process in

Linux halting the process, where the measurement is executed. Unfortunately, this cannot be prevented and can happen every time general-purpose operating systems are used.

The distribution of the gathered data is shown in Figure 7.

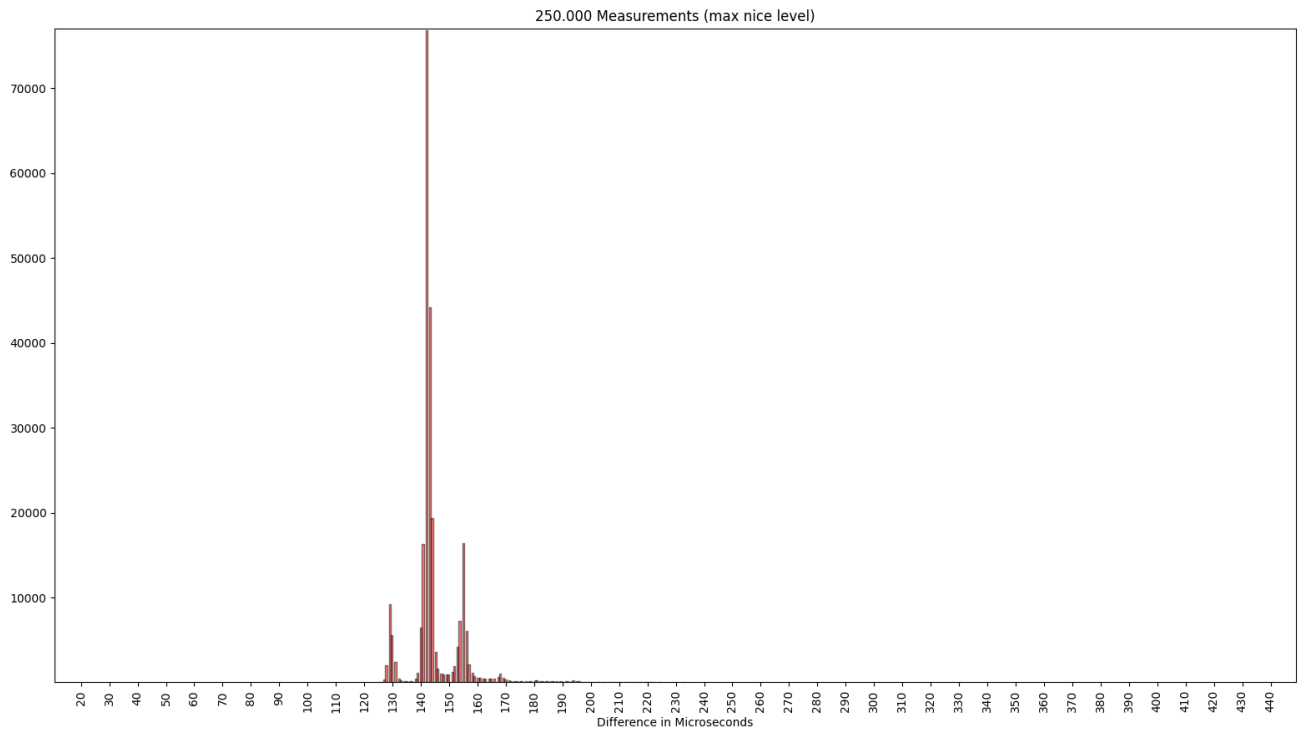


Figure 7: 250.000 measurements with Nice Level priority

The difference to the measurements without Nice Level priority increase does not seem to be significant, with most of the values settling between 140 and 150 μs once again.

List of Figures

Figure 1: Network setup	2
Figure 2: Assembled setup	4
Figure 3: 50.000 measurements without Nice Level priority	7
Figure 4: 50.000 measurements with Nice Level priority	8
Figure 5: 50.000 measurements with high CPU load	9
Figure 6: 250.000 measurements without Nice Level priority	11
Figure 7: 250.000 measurements with Nice Level priority	12

List of Tables

Table 1: IP addresses	2
Table 2: Minimum, maximum and mean values of 50.000 measurements for normal operation without Nice Level priority.....	7
Table 3: Minimum, maximum and mean values of 50.000 measurements for normal operation with Nice Level priority.....	8
Table 4: Minimum, maximum and mean values of 50.000 measurements for normal operation with high CPU load.....	9
Table 5: Minimum, maximum and mean values of 250.000 measurements for normal operation without Nice Level priority.....	10
Table 6: Minimum, maximum and mean values of 250.000 measurements for normal operation without Nice Level priority.....	11